UNIVERSIDAD PRIVADA DEL NORTE

FACULTAD DE INGENIERÍA

CARRERA: INGENIERÍA DE SISTEMAS

CURSO: TÉCNICAS DE PROGRAMACIÓN ORIENTADA A OBJETOS

DOCENTE: SEGUNDO JOSÉ CASTILLO ZUMARÁN

TEMA: SEMANA 05 – SOBRECARGA DE MÉTODOS, MANEJO DE ERRORES Y COLECCIONES

INTEGRANTES: (PARTICIPACIÓN)

- Araujo Lazo Pamela Lisette (100%)
- Cabrera Castillo Nilton Manolo (100%)
- Castro Aguilar Axel David (100%)
- Castro Reyes Leslie Melanie (100%)
- Obeso Guarniz Marcos Diego (100%)
- Palacios Alzamora Oskar Alonso (100%)

TRUJILLO – PERÚ

2026

ACTIVIDAD PRÁCTICA DE CAMPO – SEMANA 05

SOBRECARGA DE MÉTODOS, MANEJO DE ERRORES Y COLECCIONES:

- Desarrollo de práctica de sobrecarga de métodos.
- Manejo de errores mediante excepciones (try-catch, throw y throws).
- Uso de colecciones (ArrayList) para administrar objetos.
- Evidencia del aporte individual en el repositorio compartido.

Responsable de la semana: Cabrera Castillo Nilton Manolo.

Datos del trabajo:

- Repositorio compartido: https://github.com/AxelC2code/Kranklin-Labs
- Rama de trabajo: `feat/nilton-sobrecarga`
- Proyecto: `TrabajoPOO_Nilton` (Maven, Java 21, Apache NetBeans).
- Paquetes: `Ejercicios_Sesion03`, `Sobrecarga_Tienda` y `Caso_Grupal`.

DESARROLLO

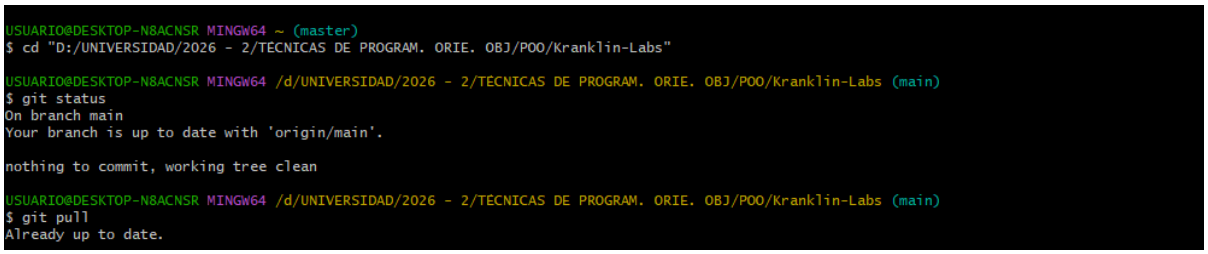
1. Ubicación en el repositorio y actualización:

Se abrió Git Bash y se ingresó a la carpeta local del repositorio compartido Kranklin-Labs, clonado previamente desde GitHub. Se revisó el estado con git status y se descargaron los últimos cambios de la rama principal con git pull.

```
cd ".../POO/Kranklin-Labs"
git status
git pull
```

El resultado mostró `Already up to date`, lo que indicó que la copia local estaba sincronizada con GitHub.

Imagen 01: Ubicación en el repositorio y actualización de la rama main

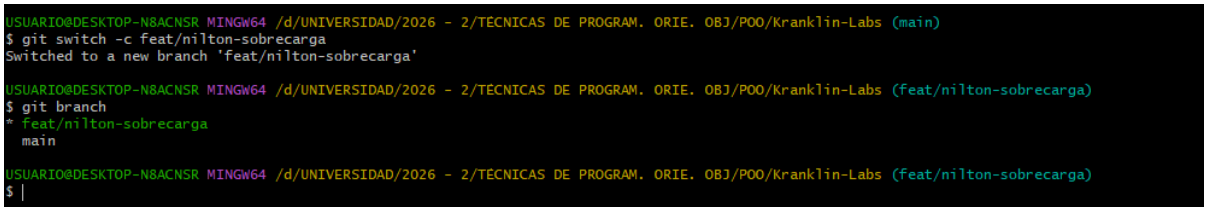


2. Creación de la rama de trabajo:

Para no modificar directamente la rama principal main, se creó la rama feat/nilton-sobrecarga, donde se registró todo el aporte individual de la semana. Con git branch se comprobó que el asterisco (*) estaba en la nueva rama.

```
git switch -c feat/nilton-sobrecarga
git branch
```

Imagen 02: Creación de la rama feat/nilton-sobrecarga



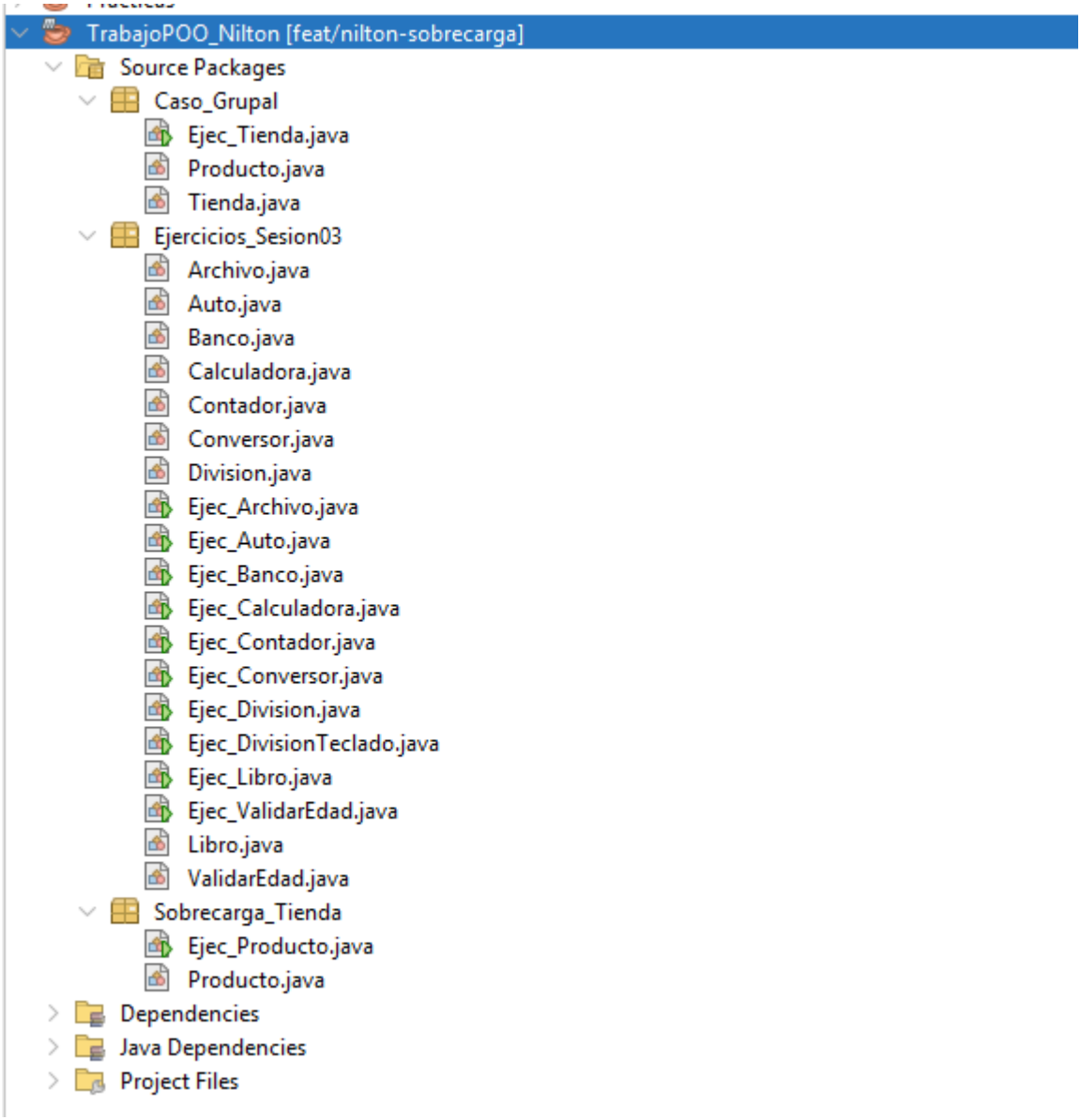
3. Creación del proyecto en NetBeans:

Dentro del repositorio se creó el proyecto Maven TrabajoPOO_Nilton, organizado en tres paquetes:

- `Ejercicios_Sesion03`: los 10 ejercicios de la Sesión 03.
- `Sobrecarga_Tienda`: sistema de productos de una tienda.
- `Caso_Grupal`: caso práctico grupal de la tienda de tecnología.

Además, se agregó un archivo .gitignore para que la carpeta target, que NetBeans genera al compilar, no se suba al repositorio.

Imagen 03: Estructura del proyecto en NetBeans



4. Ejercicios de la Sesión 03:

Se desarrollaron los 10 ejercicios de la sesión. Cada clase tiene su clase ejecutora (Ejec_…) con el método main, siguiendo la forma de trabajo del curso.

N°	Clase(s)	Tema aplicado
1	Calculadora	Sobrecarga del método sumar (dos enteros, tres enteros y dos decimales).
2	Conversor	Sobrecarga del método convertir (Celsius a Fahrenheit, km a millas, segundos a minutos).

N°	Clase(s)	Tema aplicado
3	Contador	Atributo static totalObjetos que cuenta las instancias creadas.
4	Banco	Variable static tasaInteres y métodos static para actualizarla y mostrarla.
5	Auto y Motor	Clase anidada: Auto instancia su Motor y muestra el tipo de motor.
6	Libro y Pagina	Clase interna: cada Pagina pertenece a un Libro.
7	Division	try-catch con ArithmeticException (división entre cero).
8	Ejec_DivisionTeclado	ArithmeticException e InputMismatchException con datos del usuario.
9	ValidarEdad	throw de IllegalArgumentException si la edad es menor de 18.
10	Archivo	Método con throws IOException y try-catch en el main.

Ejemplo de sobrecarga en la clase Calculadora: el mismo método sumar con distintos parámetros. Java elige qué versión ejecutar según la cantidad y el tipo de datos que recibe.

```
public int sumar(int numero1, int numero2)
public int sumar(int numero1, int numero2, int numero3)
public double sumar(double numero1, double numero2)
```

Imagen 04: Ejecución del ejercicio 1 (Calculadora)

```java
package Ejercicios_Sesion03;

public class Calculadora {

    // Sobrecarga de métodos: el mismo nombre "sumar" con distintos parámetros

    public int sumar(int numero1, int numero2) {
        return numero1 + numero2;
    }

    public int sumar(int numero1, int numero2, int numero3) {
        return numero1 + numero2 + numero3;
    }

    public double sumar(double numero1, double numero2) {
        return numero1 + numero2;
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_Calculadora {

    public static void main(String[] args) {

        Calculadora calc = new Calculadora();

        System.out.println("----- EJERCICIO 1: CALCULADORA -----");
        System.out.println("Suma de dos enteros (5 + 3): " + calc.sumar(5, 3));
        System.out.println("Suma de tres enteros (5 + 3 + 2): " + calc.sumar(5, 3, 2));
        System.out.println("Suma de dos decimales (2.5 + 4.3): " + calc.sumar(2.5, 4.3));
    }
}
```

Imagen 05: Ejecución del ejercicio 2 (Conversor)

```java
package Ejercicios_Sesion03;

public class Conversor {

    // Sobrecarga: Java elige el método según el tipo de dato que recibe

    // Celsius a Fahrenheit (recibe double)
    public double convertir(double celsius) {
        return celsius * 9 / 5 + 32;
    }

    // Kilómetros a millas (recibe float)
    public double convertir(float kilometros) {
        return kilometros * 0.621371;
    }

    // Segundos a minutos y segundos (recibe int)
    public String convertir(int segundos) {
        int minutos = segundos / 60;
        int segundosRestantes = segundos % 60;
        return minutos + " minutos y " + segundosRestantes + " segundos";
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_Conversor {

    public static void main(String[] args) {

        Conversor conv = new Conversor();

        double celsius = 25.0;
        float kilometros = 10.0f;
        int segundos = 125;

        System.out.println("----- EJERCICIO 2: CONVERSOR -----");
        System.out.println(celsius + " grados Celsius = " + conv.convertir(celsius) + " grados Fahrenheit");
        System.out.println(kilometros + " km = " + String.format("%.2f", conv.convertir(kilometros)) + " millas");
        System.out.println(segundos + " segundos = " + conv.convertir(segundos));
    }
}
```

Imagen 06: Ejecución del ejercicio 3 (Contador)

```java
package Ejercicios_Sesion03;

public class Contador {

    // static: hay una sola variable para toda la clase y la comparten todos los objetos
    private static int totalObjetos = 0;

    public Contador() {
        totalObjetos++;
    }

    public static void mostrarTotal() {
        System.out.println("Total de objetos creados: " + totalObjetos);
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_Contador {

    public static void main(String[] args) {

        System.out.println("----- EJERCICIO 3: CONTADOR -----");

        Contador obj1 = new Contador();
        Contador obj2 = new Contador();
        Contador obj3 = new Contador();
        Contador.mostrarTotal();

        System.out.println("Se crea un objeto mas...");
        Contador obj4 = new Contador();
        Contador.mostrarTotal();
    }
}
```

Imagen 07: Ejecución del ejercicio 4 (Banco)

```java
package Ejercicios_Sesion03;

public class Banco {

    // static: la tasa es la misma para todo el banco
    private static double tasaInteres = 5.0;

    public static void actualizarTasa(double nuevaTasa) {
        if (nuevaTasa >= 0) {
            tasaInteres = nuevaTasa;
            System.out.println("Tasa actualizada correctamente.");
        } else {
            System.out.println("Error: la tasa no puede ser negativa.");
        }
    }

    public static void mostrarTasa() {
        System.out.println("Tasa de interes actual: " + tasaInteres + "%");
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_Banco {

    public static void main(String[] args) {

        System.out.println("----- EJERCICIO 4: BANCO -----");

        // Los métodos static se llaman con el nombre de la clase, sin crear objetos
        Banco.mostrarTasa();

        Banco.actualizarTasa(8.5);
        Banco.mostrarTasa();

        Banco.actualizarTasa(-2);
        Banco.mostrarTasa();
    }
}
```

Imagen 08: Ejecución del ejercicio 5 (Auto y Motor)

```java
package Ejercicios_Sesion03;

public class Auto {

    private String marca;
    private Motor motor;

    public Auto(String marca, String tipoMotor) {
        this.marca = marca;
        this.motor = new Motor(tipoMotor); // desde Auto se instancia Motor
    }

    public void mostrarMotor() {
        System.out.println("Auto: " + marca + " | Tipo de motor: " + motor.obtenerTipo());
    }

    // Clase anidada: Motor está declarada dentro de Auto
    public static class Motor {

        private String tipo;

        public Motor(String tipo) {
            this.tipo = tipo;
        }

        public String obtenerTipo() {
            return tipo;
        }
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_Auto {

    public static void main(String[] args) {

        System.out.println("----- EJERCICIO 5: AUTO Y MOTOR -----");

        Auto auto1 = new Auto("Ford Mustang", "V8");
        Auto auto2 = new Auto("Tesla Model 3", "Electrico");

        auto1.mostrarMotor();
        auto2.mostrarMotor();
    }
}
```

Imagen 09: Ejecución del ejercicio 6 (Libro y Pagina)

```java
package Ejercicios_Sesion03;

public class Libro {

    private String titulo;

    public Libro(String titulo) {
        this.titulo = titulo;
    }

    // Clase interna: cada Pagina pertenece a un objeto Libro
    public class Pagina {

        private int numero;
        private String contenido;

        public Pagina(int numero, String contenido) {
            this.numero = numero;
            this.contenido = contenido;
        }

        public void mostrarContenido() {
            // La clase interna puede usar los atributos del Libro (titulo)
            System.out.println(titulo + " - Pagina " + numero + ": " + contenido);
        }
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_Libro {

    public static void main(String[] args) {

        System.out.println("----- EJERCICIO 6: LIBRO Y PAGINA -----");

        Libro libro = new Libro("Java para principiantes");

        // Para crear una Pagina primero se necesita un objeto Libro
        Libro.Pagina pagina1 = libro.new Pagina(1, "Que es la programacion orientada a objetos");
        Libro.Pagina pagina2 = libro.new Pagina(2, "Clases, objetos y atributos");

        pagina1.mostrarContenido();
        pagina2.mostrarContenido();
    }
}
```

Imagen 10: Ejecución del ejercicio 7 (Division)

```java
package Ejercicios_Sesion03;

public class Division {

    public void dividir(int numero1, int numero2) {

        try {
            int resultado = numero1 / numero2;
            System.out.println(numero1 + " / " + numero2 + " = " + resultado);

        } catch (ArithmeticException e) {
            System.out.println("Error: no se puede dividir " + numero1 + " entre cero.");
        }
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_Division {

    public static void main(String[] args) {

        System.out.println("----- EJERCICIO 7: DIVISION -----");

        Division div = new Division();

        div.dividir(20, 4);
        div.dividir(15, 0);
    }
}
```

Imagen 11: Ejecución del ejercicio 8 (división con datos del usuario)

```java
package Ejercicios_Sesion03;

import java.util.InputMismatchException;
import java.util.Scanner;

public class Ejec_DivisionTeclado {

    public static void main(String[] args) {

        Scanner teclado = new Scanner(System.in);
        String respuesta;

        System.out.println("----- EJERCICIO 8: DIVISION CON DATOS DEL USUARIO -----");

        do {
            try {
                System.out.print("Ingrese el primer numero: ");
                int numero1 = teclado.nextInt();

                System.out.print("Ingrese el segundo numero: ");
                int numero2 = teclado.nextInt();

                int resultado = numero1 / numero2;
                System.out.println("Resultado: " + resultado);

            } catch (ArithmeticException e) {
                System.out.println("Error: no se puede dividir entre cero.");

            } catch (InputMismatchException e) {
                System.out.println("Error: debe ingresar solo numeros enteros.");
            }

            teclado.nextLine(); // limpia lo que quedó escrito en la línea
            System.out.print("Desea hacer otra division? (s/n): ");
            respuesta = teclado.nextLine();

        } while (respuesta.equalsIgnoreCase("s"));

        System.out.println("Programa terminado.");
    }
}
```

Imagen 12: Ejecución del ejercicio 9 (ValidarEdad)

```java
package Ejercicios_Sesion03;

public class ValidarEdad {

    public void verificarEdad(int edad) {

        if (edad < 18) {
            throw new IllegalArgumentException("La edad " + edad + " no es valida, debe ser mayor o igual a 18.");
        }

        System.out.println("Edad " + edad + ": edad valida.");
    }
}
```

```java
package Ejercicios_Sesion03;

public class Ejec_ValidarEdad {

    public static void main(String[] args) {

        System.out.println("----- EJERCICIO 9: VALIDAR EDAD -----");

        ValidarEdad validar = new ValidarEdad();
        int[] edades = {25, 18, 15};

        for (int edad : edades) {
            try {
                validar.verificarEdad(edad);

            } catch (IllegalArgumentException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

Imagen 13: Ejecución del ejercicio 10 (Archivo)

```java
package Ejercicios_Sesion03;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class Archivo {

    // throws IOException: el método avisa que puede fallar y quien lo llame debe manejar el error
    public void leerArchivo(String nombreArchivo) throws IOException {

        BufferedReader lector = new BufferedReader(new FileReader(nombreArchivo));
        String linea;

        System.out.println("Contenido de " + nombreArchivo + ":");

        while ((linea = lector.readLine()) != null) {
            System.out.println("  " + linea);
        }

        lector.close();
    }
}
```

```java
package Ejercicios_Sesion03;

import java.io.IOException;

public class Ejec_Archivo {

    public static void main(String[] args) {

        System.out.println("----- EJERCICIO 10: LEER ARCHIVO -----");

        Archivo archivo = new Archivo();

        // Caso 1: el archivo existe
        try {
            archivo.leerArchivo("datos.txt");

        } catch (IOException e) {
            System.out.println("Error: no se pudo leer el archivo datos.txt");
        }

        // Caso 2: el archivo no existe
        try {
            archivo.leerArchivo("no_existe.txt");

        } catch (IOException e) {
            System.out.println("Error: no se pudo leer el archivo no_existe.txt");
        }
    }
}
```

5. Sistema de productos de una tienda (sobrecarga):

Se implementó la clase Producto con sobrecarga de constructores, para registrar un producto de tres formas según la información disponible, y sobrecarga del método calcularPrecioVenta, para calcular el precio de venta de tres maneras.

```
public Producto(String codigo, String nombre)
public Producto(String codigo, String nombre, double precio)
public Producto(String codigo, String nombre, double precio, int stock)

public double calcularPrecioVenta(double porcentajeDescuento)
public double calcularPrecioVenta(float descuentoSoles)
public double calcularPrecioVenta(double porcentajeDescuento, int cantidad)
```

El porcentaje de descuento se recibe como double (10.0) y el descuento fijo en soles como float (50.0f). Por el tipo de dato, Java distingue qué versión del método debe usar.

Imagen 14: Ejecución del sistema de productos de una tienda

```java
package Sobrecarga_Tienda;

public class Producto {

    private String codigo;
    private String nombre;
    private double precio;
    private int stock;

    // Sobrecarga de constructores: el producto se puede registrar de 3 formas

    // Solamente con código y nombre
    public Producto(String codigo, String nombre) {
        this(codigo, nombre, 0, 0); // llama al constructor completo
    }

    // Con código, nombre y precio
    public Producto(String codigo, String nombre, double precio) {
        this(codigo, nombre, precio, 0);
    }

    // Con código, nombre, precio y stock
    public Producto(String codigo, String nombre, double precio, int stock) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public String getCodigo() {
        return codigo;
    }

    public void setCodigo(String codigo) {
        this.codigo = codigo;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public double getPrecio() {
        return precio;
    }
```

```
    public void setPrecio(double precio) {
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }

    // Sobrecarga de métodos: el precio de venta se calcula de 3 formas

    // 1. Aplicando un porcentaje de descuento (recibe double)
    public double calcularPrecioVenta(double porcentajeDescuento) {
        return precio - precio * porcentajeDescuento / 100;
    }

    // 2. Aplicando un descuento fijo en soles (recibe float)
    public double calcularPrecioVenta(float descuentoSoles) {
        return precio - descuentoSoles;
    }

    // 3. Aplicando descuento y cantidad de unidades
    public double calcularPrecioVenta(double porcentajeDescuento, int cantidad) {
        return calcularPrecioVenta(porcentajeDescuento) * cantidad;
    }

    public void mostrarInformacion() {
        System.out.println("Codigo: " + codigo + " | Nombre: " + nombre
                + " | Precio: S/ " + String.format("%.2f", precio) + " | Stock: " + stock);
    }
}
```

```
package Sobrecarga_Tienda;

public class Ejec_Producto {

    public static void main(String[] args) {

        System.out.println("----- SISTEMA DE PRODUCTOS DE UNA TIENDA -----");

        // Cada producto usa un constructor distinto
        Producto producto1 = new Producto("P001", "Mouse inalambrico");
        Producto producto2 = new Producto("P002", "Teclado mecanico", 150.00);
        Producto producto3 = new Producto("P003", "Monitor 24 pulgadas", 600.00, 10);

        System.out.println();
        System.out.println("--- Registro de productos (sobrecarga de constructores) ---");
        producto1.mostrarInformacion();
        producto2.mostrarInformacion();
        producto3.mostrarInformacion();

        // 10.0 es double (porcentaje) y 50.0f es float (soles):
        // por el tipo de dato Java sabe qué versión del método usar
        System.out.println();
        System.out.println("--- Precio de venta del " + producto3.getNombre() + " (sobrecarga de metodos) ---");
        System.out.println("Precio normal: S/ " + String.format("%.2f", producto3.getPrecio()));
        System.out.println("Con 10% de descuento: S/ " + String.format("%.2f", producto3.calcularPrecioVenta(10.0)));
        System.out.println("Con S/ 50 de descuento fijo: S/ " + String.format("%.2f", producto3.calcularPrecioVenta(50.0f)));
        System.out.println("3 unidades con 10% de descuento: S/ " + String.format("%.2f", producto3.calcularPrecioVenta(10.0, 3)));
    }
}
```

6. Caso práctico grupal (manejo de errores y colecciones):

Se diseñó la clase Producto de una tienda de tecnología (código, nombre, categoría, precio y stock), con getters y setters para consultar y modificar sus datos, y los métodos calcularValorStock, calcularPrecioVenta, calcularImporte y mostrarInformacion. Las reglas del negocio se controlan con excepciones:

- El precio no puede ser negativo.
- El stock no puede ser negativo.
- El descuento debe estar entre 0% y 100%.
- La cantidad a vender no puede ser mayor que el stock disponible.

```
private void validarPrecio(double precio) {
    if (precio < 0) {
        throw new IllegalArgumentException(
                "El precio no puede ser negativo.");
    }
}
```

Para las colecciones se creó la clase Tienda, que guarda los productos en un ArrayList<Producto> y permite registrarlos sin repetir el código, buscarlos, listarlos y calcular el valor total del inventario. En Ejec_Tienda cada error se captura con try-catch, de modo que el programa avisa el problema y continúa.

```
private ArrayList<Producto> productos;

try {
    laptop.calcularImporte(20);
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
}
```

Imagen 15: Ejecución del caso grupal: registro, consulta y operaciones

```java
package Caso_Grupal;

import java.util.ArrayList;

public class Tienda {

    private String nombre;

    // Colección: lista donde se guardan todos los productos de la tienda
    private ArrayList<Producto> productos;

    public Tienda(String nombre) {
        this.nombre = nombre;
        this.productos = new ArrayList<Producto>();
    }

    public void agregarProducto(Producto producto) {
        for (Producto p : productos) {
            if (p.getCodigo().equals(producto.getCodigo())) {
                throw new IllegalArgumentException("Ya existe un producto con el codigo " + producto.getCodigo() + ".");
            }
        }
        productos.add(producto);
        System.out.println("Producto registrado: " + producto.getNombre());
    }

    public Producto buscarProducto(String codigo) {
        for (Producto p : productos) {
            if (p.getCodigo().equals(codigo)) {
                return p;
            }
        }
        throw new IllegalArgumentException("No existe un producto con el codigo " + codigo + ".");
    }

    public void mostrarProductos() {
        System.out.println("--- Productos de " + nombre + " (" + productos.size() + " en total) ---");
        for (Producto p : productos) {
            p.mostrarInformacion();
        }
    }

    public double calcularValorTotalStock() {
        double total = 0;
        for (Producto p : productos) {
            total = total + p.calcularValorStock();
        }
        return total;
    }
```

```java
package Caso_Grupal;

public class Producto {

    private String codigo;
    private String nombre;
    private String categoria;
    private double precio;
    private int stock;

    // Sobrecarga de constructores

    // Al inicio solo se conoce el código y el nombre
    public Producto(String codigo, String nombre) {
        this(codigo, nombre, "Sin categoria", 0, 0);
    }

    // Se conoce el código, nombre y precio
    public Producto(String codigo, String nombre, double precio) {
        this(codigo, nombre, "Sin categoria", precio, 0);
    }

    // Se tiene toda la información, incluyendo el stock
    public Producto(String codigo, String nombre, String categoria, double precio, int stock) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.categoria = categoria;
        validarPrecio(precio); // se valida desde el inicio
        validarStock(stock);
        this.precio = precio;
        this.stock = stock;
    }

    // Getters y setters: consultar y modificar la información

    public String getCodigo() {
        return codigo;
    }

    public void setCodigo(String codigo) {
        this.codigo = codigo;
    }

    public String getNombre() {
        return nombre;
    }
```

```
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getCategoria() {
        return categoria;
    }

    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        validarPrecio(precio);
        this.precio = precio;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        validarStock(stock);
        this.stock = stock;
    }

    // Validaciones: el precio y el stock no pueden ser negativos

    private void validarPrecio(double precio) {
        if (precio < 0) {
            throw new IllegalArgumentException("El precio no puede ser negativo.");
        }
    }

    private void validarStock(int stock) {
        if (stock < 0) {
            throw new IllegalArgumentException("El stock no puede ser negativo.");
        }
    }

```

```
    // Operaciones

    // Valor del stock = precio x unidades disponibles
    public double calcularValorStock() {
        return precio * stock;
    }

    // Precio de venta aplicando un descuento entre 0% y 100%
    public double calcularPrecioVenta(double porcentajeDescuento) {
        if (porcentajeDescuento < 0 || porcentajeDescuento > 100) {
            throw new IllegalArgumentException("El descuento debe estar entre 0% y 100%.");
        }
        return precio - precio * porcentajeDescuento / 100;
    }

    // Importe de una cantidad de unidades
    public double calcularImporte(int cantidad) {
        validarCantidad(cantidad);
        return precio * cantidad;
    }

    // Sobrecarga: importe de una cantidad de unidades con descuento
    public double calcularImporte(int cantidad, double porcentajeDescuento) {
        validarCantidad(cantidad);
        return calcularPrecioVenta(porcentajeDescuento) * cantidad;
    }

    // La cantidad a vender no puede ser mayor que el stock disponible
    private void validarCantidad(int cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad debe ser mayor que 0.");
        }
        if (cantidad > stock) {
            throw new IllegalArgumentException("No hay stock suficiente: se piden " + cantidad
                    + " unidades y solo hay " + stock + ".");
        }
    }

    public void mostrarInformacion() {
        System.out.println("[" + codigo + "] " + nombre + " | Categoria: " + categoria
                + " | Precio: S/ " + String.format("%.2f", precio) + " | Stock: " + stock);
    }
}
```

Imagen 16: Ejecución del caso grupal: validaciones

```java
package Caso_Grupal;

public class Ejec_Tienda {

    public static void main(String[] args) {

        Tienda tienda = new Tienda("TecnoStore");

        System.out.println("----- CASO PRACTICO: TIENDA DE TECNOLOGIA -----");

        // 1. Registrar productos con los 3 constructores y guardarlos en la lista
        System.out.println();
        System.out.println("1. Registro de productos");
        tienda.agregarProducto(new Producto("T001", "Audifonos Bluetooth"));
        tienda.agregarProducto(new Producto("T002", "Memoria USB 64GB", 35.90));
        tienda.agregarProducto(new Producto("T003", "Laptop Lenovo IdeaPad", "Computo", 2500.00, 8));
        tienda.agregarProducto(new Producto("T004", "Mouse Logitech", "Accesorios", 45.50, 30));
        tienda.mostrarProductos();

        // 2. Consultar y modificar un producto
        System.out.println();
        System.out.println("2. Consultar y modificar un producto");
        Producto memoria = tienda.buscarProducto("T002");
        System.out.print("Antes:   ");
        memoria.mostrarInformacion();
        memoria.setCategoria("Almacenamiento");
        memoria.setPrecio(32.50);
        memoria.setStock(50);
        System.out.print("Despues: ");
        memoria.mostrarInformacion();

        // 3. Operaciones
        System.out.println();
        System.out.println("3. Operaciones con la laptop");
        Producto laptop = tienda.buscarProducto("T003");
        System.out.println("Valor del stock: S/ " + formato(laptop.calcularValorStock()));
        System.out.println("Precio con 15% de descuento: S/ " + formato(laptop.calcularPrecioVenta(15)));
        System.out.println("Importe de 2 unidades: S/ " + formato(laptop.calcularImporte(2)));
        System.out.println("Importe de 2 unidades con 15% de descuento: S/ " + formato(laptop.calcularImporte(2, 15)));
        System.out.println("Valor total del inventario: S/ " + formato(tienda.calcularValorTotalStock()));

        // 4. Reglas del sistema: cada error se captura con try-catch
        System.out.println();
        System.out.println("4. Validaciones (manejo de errores)");
```

```
        System.out.println();
        System.out.println("4. Validaciones (manejo de errores)");

        try {
            laptop.setPrecio(-100);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            tienda.agregarProducto(new Producto("T005", "Teclado Redragon", "Accesorios", 120.00, -5));
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            laptop.calcularPrecioVenta(120);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            laptop.calcularImporte(20);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            tienda.agregarProducto(new Producto("T001", "Parlante JBL"));
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        try {
            tienda.buscarProducto("T999");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }

        System.out.print("La laptop no cambio: ");
        laptop.mostrarInformacion();
    }

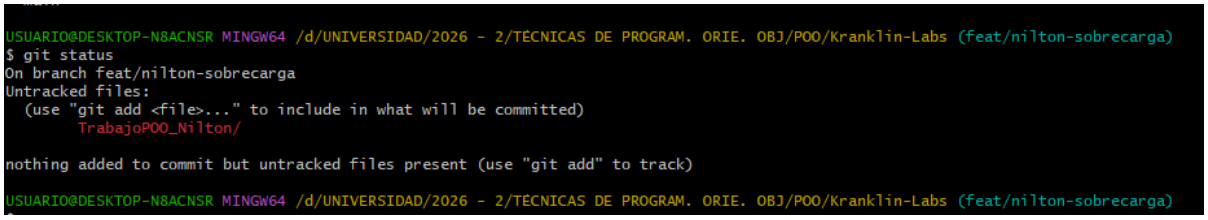
    private static String formato(double valor) {
        return String.format("%.2f", valor);
    }
}
```

7. Verificación de los archivos nuevos:

Antes de registrar el trabajo se ejecutó git status. Git mostró la carpeta TrabajoPOO_Nilton/ en rojo, como archivo sin seguimiento (untracked).

```
git status
```

Imagen 17: Archivos nuevos detectados por Git

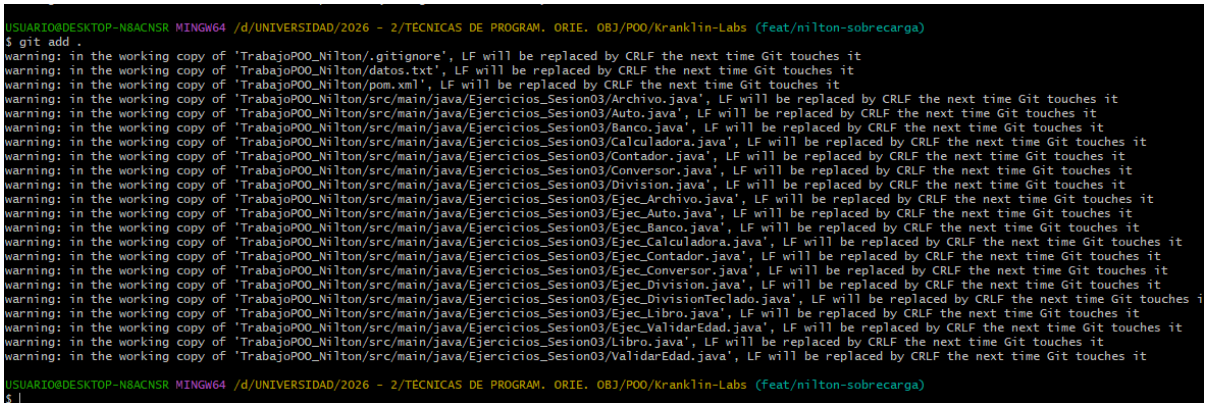


8. Preparación de los archivos (git add):

Con git add se agregaron los archivos al área de preparación. Git mostró avisos (warning) sobre el fin de línea LF y CRLF: no son errores, solo indican que Git ajustará los saltos de línea al formato de Windows.

```
git add .
```

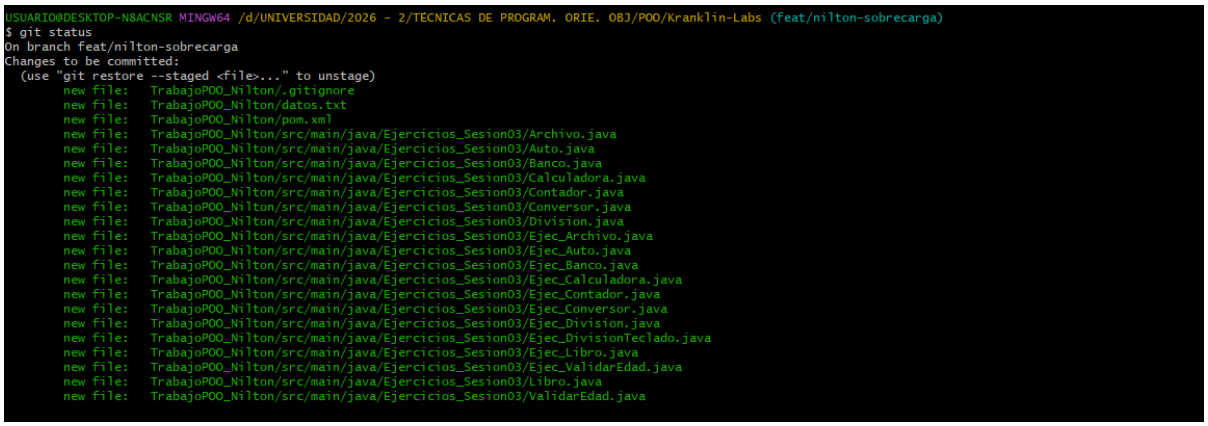
Imagen 18: Archivos agregados con git add



9. Verificación del área de preparación:

Al volver a ejecutar git status, los 22 archivos aparecieron en verde como new file, listos para el commit.

Imagen 19: Archivos listos para el commit

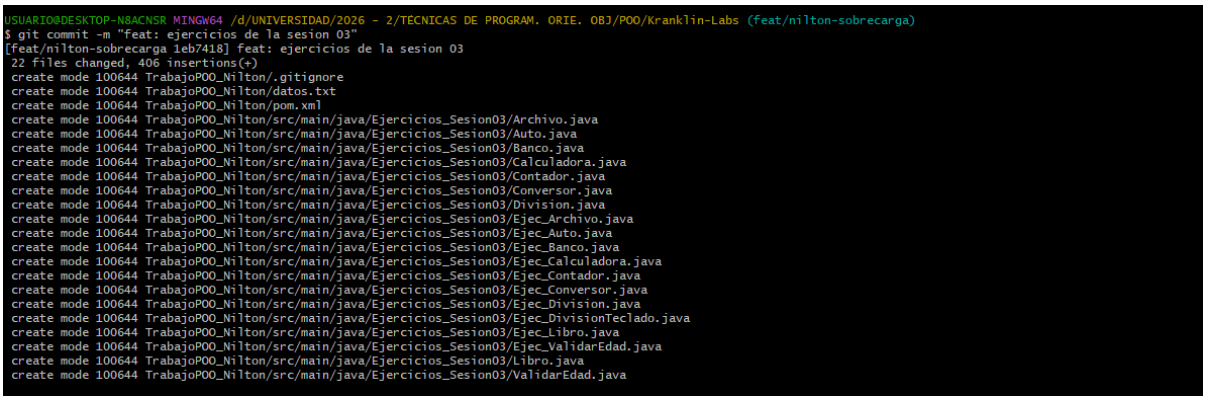


10. Primer commit: ejercicios de la Sesión 03:

```
git commit -m "feat: ejercicios de la sesion 03"
```

Se generó el commit 1eb7418 con 22 archivos: el proyecto, el .gitignore y los 10 ejercicios con sus clases ejecutoras.

Imagen 20: Primer commit (1eb7418)



11. Segundo commit: sistema de productos de una tienda:

Se repitió el ciclo status, add y commit para el paquete Sobrecarga_Tienda. Esta vez no aparecieron avisos de fin de línea.

```
git status
git add .
git commit -m "feat: sobrecarga de constructores y metodos en Producto
(tienda)"
```

Imagen 21: Paquete Sobrecarga_Tienda detectado por Git

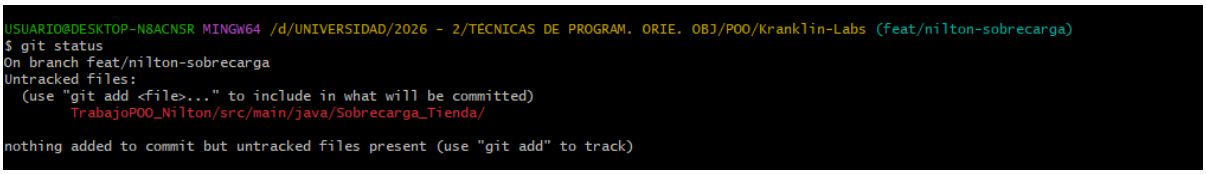
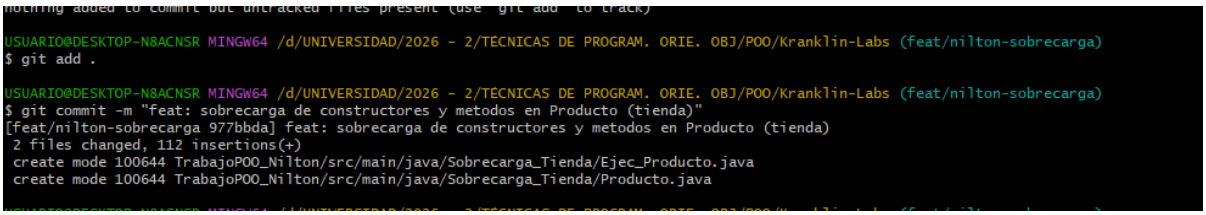


Imagen 22: Segundo commit (977bbda)

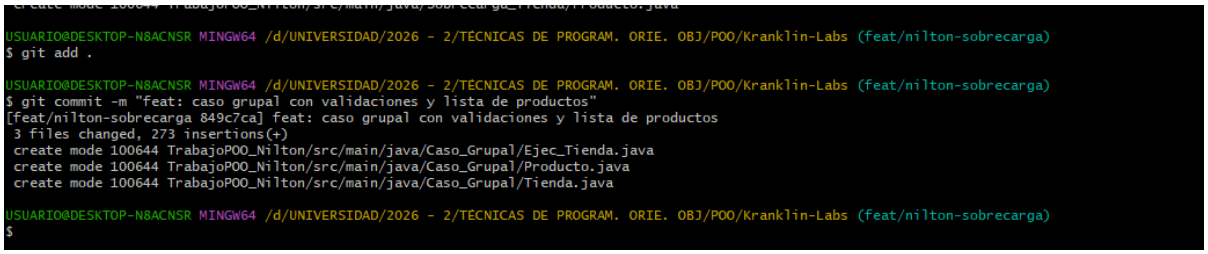


12. Tercer commit: caso práctico grupal:

```
git add .
git commit -m "feat: caso grupal con validaciones y lista de productos"
```

Se generó el commit 849c7ca con las clases Producto, Tienda y Ejec_Tienda del paquete Caso_Grupal.

Imagen 23: Tercer commit (849c7ca)

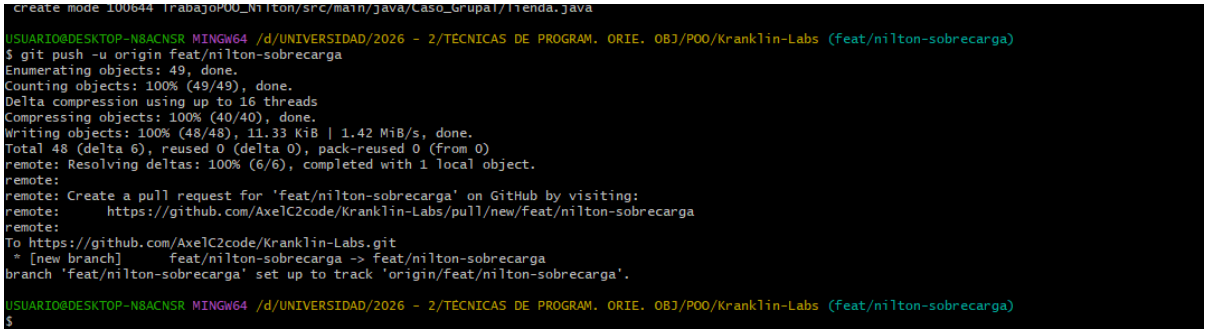


13. Subida de la rama a GitHub (git push):

```
git push -u origin feat/nilton-sobrecarga
```

La rama se subió al repositorio compartido. Git confirmó la nueva rama ([new branch]) y mostró el enlace para crear el Pull Request. La opción -u deja conectada la rama local con la de GitHub.

Imagen 24: Subida de la rama feat/nilton-sobrecarga a GitHub

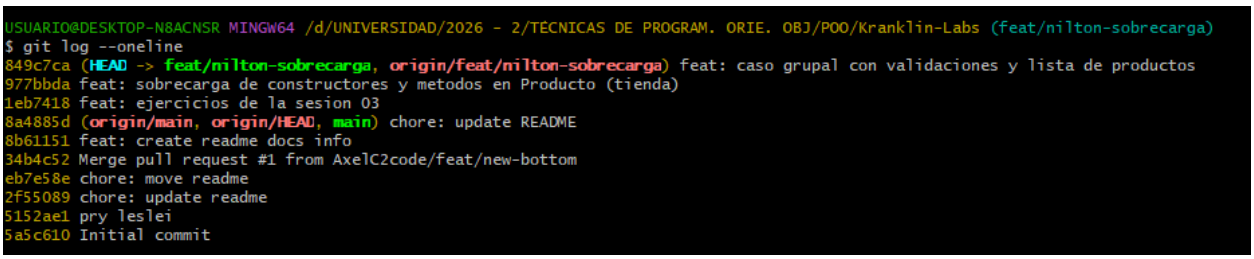


14. Historial de commits:

```
git log --oneline
```

El historial muestra los tres commits del aporte individual sobre el último commit de la rama principal.

Imagen 25: Historial de commits de la rama



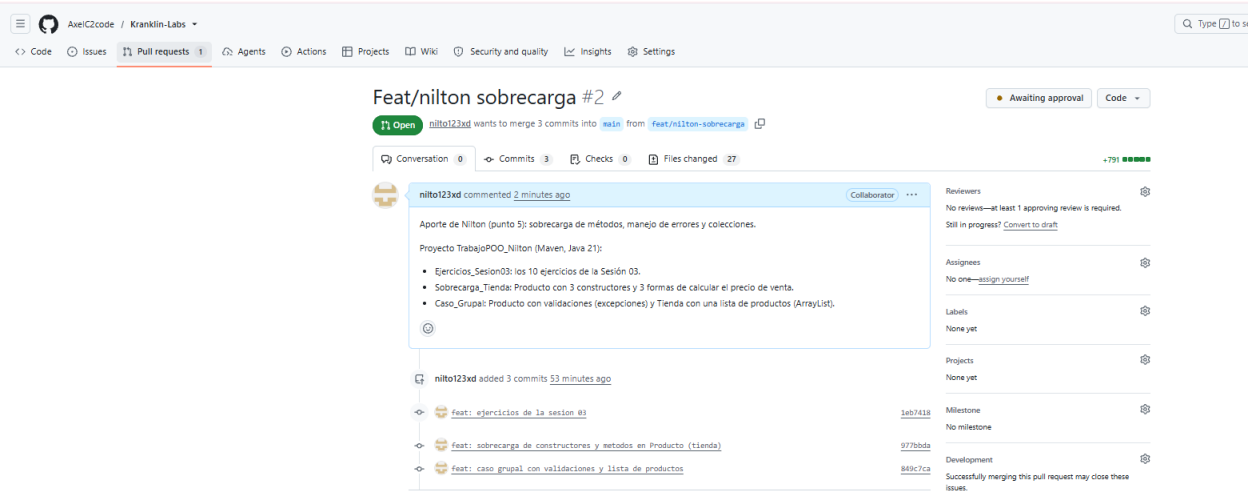
15. Creación del Pull Request:

Se creó un Pull Request para solicitar la integración de los cambios a la rama principal.

- Rama origen: `feat/nilton-sobrecarga`
- Rama destino: `main`

El Pull Request permite que el equipo revise el aporte antes de integrarlo al proyecto.

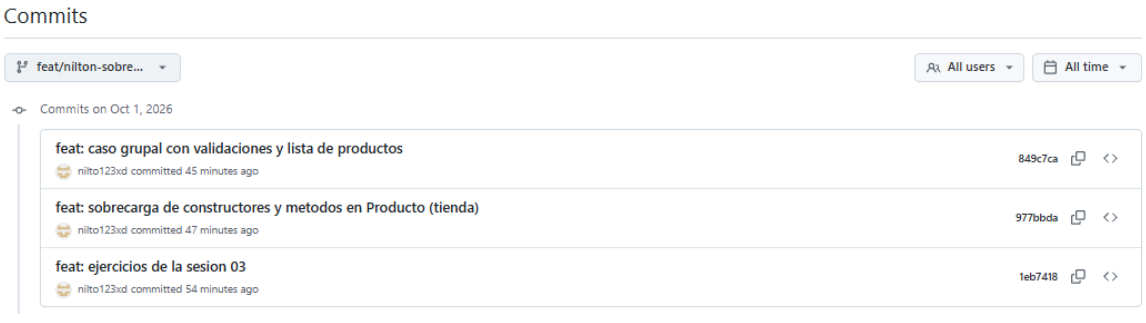
Imagen 26: Pull Request creado en GitHub



16. Evidencia del aporte individual en GitHub:

En la pestaña Commits de la rama feat/nilton-sobrecarga, los tres commits aparecen registrados con la cuenta de GitHub nilto123xd, lo que evidencia el aporte individual en el repositorio compartido.

Imagen 27: Commits del aporte individual en GitHub



CONCLUSIONES

- La sobrecarga permite usar el mismo nombre de método o constructor con distintos parámetros, lo que hace el código más claro y flexible.

- Las excepciones (try-catch, throw y throws) permiten controlar los errores y hacer cumplir las reglas del negocio sin que el programa se detenga.
- Las colecciones como ArrayList facilitan guardar, buscar y recorrer varios objetos del mismo tipo.
- Trabajar en una rama propia, con commits pequeños y mensajes claros, deja evidencia del aporte individual en el repositorio compartido.